WireGuard VPN User Authentication

Projeto de Identificação, Autenticação e Autorização

Mestrado em Cibersegurança — DETI — Universidade de Aveiro

Ano Letivo 2025/2026

Nome Autor 1 — XXXXX
Nome Autor 2 — XXXXX
Nome Autor 3 — XXXXX

25 de maio de 2026

Índice

1 Introdução **3**

2 Arquitetura **3**
2.1 Componentes 3
2.2 Diagrama de componentes 4

3 Modelo de confiança **4**

4 Fluxo de autenticação ponto-a-ponto **4**

5 Túnel Noise-IK **5**

6 Proxy HTTP local **6**

7 Fatores de autenticação (MFA) **6**
7.1 Palavra-passe — desafio-resposta 6
7.2 TOTP 7
7.3 FIDO2 / WebAuthn 7

8 Orquestração PAM **7**
8.1 Módulo PAM customizado 7
8.2 Funcionamento 7
8.3 Contrato de conversação 8
8.4 Mapeamento de tiers 8

9 Autenticação baseada em risco (RBA) **8**

10 Camada de dados **8**

11 Robustecimento de segurança **9**

12 Testes e validação **9**
12.1 Ambiente de teste 9
12.2 Registo de utilizador (Helpdesk) 10
12.3 Fluxo de autenticação completo 10
12.4 Túnel WireGuard ativo 11
12.5 Tiers de risco diferentes 12
12.6 Bloqueio de conta 12
12.7 Dados cifrados em repouso 12

13 Funcionalidades não implementadas **13**

14 Conclusão **13**

15 Contribuições **13**

16 Código de terceiros **13**

1 Introdução

O presente projeto tem como objetivo a conceção e implementação de um protocolo seguro de autenticação de utilizadores para WireGuard VPN. O WireGuard, por defeito, autentica pares através de chaves assimétricas pré-configuradas, sem qualquer mecanismo de *login* associado ao utilizador. Este projeto adiciona uma camada de autenticação que combina *Multi-Factor Authentication* (MFA) com *Risk-Based Authentication* (RBA), orquestrada por PAM, sobre um fluxo OAuth 2.0 Device Authorization Grant (RFC 8628).

Após autenticação bem-sucedida, o *gateway* provisiona dinamicamente os parâmetros WireGuard necessários (chave privada do cliente, IP do túnel e chave pré-partilhada) e o túnel VPN é estabelecido automaticamente. A única raiz de confiança é a chave pública WireGuard do servidor — não existem certificados, CAs ou PKI.

2 Arquitetura

O sistema é composto por quatro processos distribuídos por três contentores Docker, mais o cliente que corre na máquina do utilizador.

2.1 Componentes

UserAuthn (`client/userauthn.py`)
: Aplicação cliente que inicia o túnel Noise-IK para o SrvAuthn, despoleta o fluxo OAuth Device Grant e aplica a configuração WireGuard recebida.

SrvAuthn (`server/srvauthn.py`)
: Proxy de autenticação e provisionador WireGuard no gateway. Escuta na porta TCP 9999 (Noise-IK) e opera em `network_mode: host` para gerir diretamente a interface `wg0` do kernel.

IdP (`idp/app.py`)
: Servidor OAuth 2.0 (Flask) que implementa o Device Authorization Grant e gere todo o fluxo MFA via PAM. Escuta na porta 8080.

Helpdesk (`idp/helpdesk/app.py`)
: Interface web interna (loopback `127.0.0.1:8081`) para registo de utilizadores e gestão de credenciais, sem qualquer validação de segurança — conforme indicado na especificação.

`pam_vpnmfa.so` (`idp/pam_iaa/src/pam_vpnmfa.c`)
: Módulo PAM customizado que lê a pontuação de risco e decide quais fatores de autenticação exigir, comunicando com o IdP via função de conversação PAM.

2.2 Diagrama de componentes

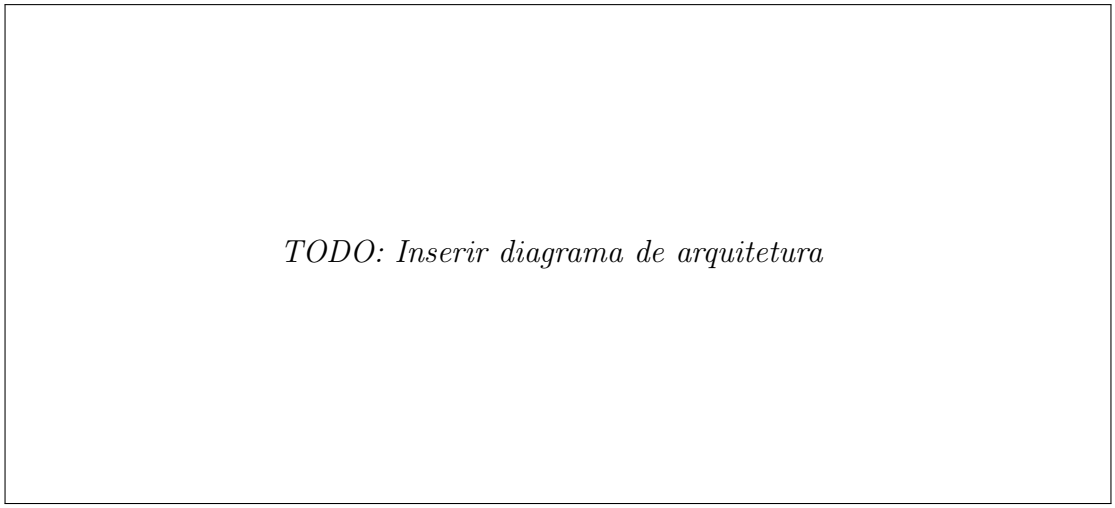


Figura 1: Diagrama de componentes do sistema.

3 Modelo de confiança

A única raiz de confiança é a chave pública WireGuard do gateway — um valor Curve25519 de 32 bytes. Esta chave desempenha dois papéis:

1. **Chave estática do responder Noise-IK** para o túnel de autenticação. O *handshake* falha se o servidor não provar posse da chave privada correspondente.
2. **Chave estática WireGuard** para o plano de dados, usada pelo kernel após o túnel ser provisionado.

Não existe TLS em nenhum ponto do sistema. O browser acede ao IdP através de um proxy HTTP local que reutiliza o mesmo túnel Noise-IK (secção 6), eliminando a necessidade de certificados adicionais.

4 Fluxo de autenticação ponto-a-ponto

O fluxo completo, desde o arranque do cliente até ao túnel VPN ativo:

1. O UserAuthn gera uma PSK localmente (`wg genpsk`).
2. Abre um túnel Noise-IK (TCP 9999) para o SrvAuthn, autenticado pela chave pública WireGuard do servidor.
3. O SrvAuthn inicia o fluxo OAuth Device Grant junto do IdP (`POST /oauth/device/code`).
4. O UserAuthn recebe o `user_code` e o `verification_uri` e abre o browser automaticamente.
5. O browser liga-se ao proxy HTTP local (`127.0.0.1:18080`), que encaminha o tráfego pelo túnel Noise-IK até ao IdP.
6. O utilizador submete o código e inicia o fluxo MFA:

- O IdP calcula a pontuação de risco (RBA).
- O *PAM bridge* invoca `pam_authenticate("iaa-vpn", ...)`.
- O módulo `pam_vpnmfa.so` determina o tier (L1/L2/L3) e emite os fatores necessários via conversação PAM.
- Para cada fator, o IdP apresenta o formulário correspondente ao browser e submete o resultado (`OK`/`FAIL`/`UNAVAILABLE`).

7. Após sucesso MFA, o IdP emite um *access token*.
8. O SrvAuthn obtém o token via polling, gera o par de chaves WG do cliente, aloca um IP do *subnet* `10.10.10.0/24` e executa `wg set wg0 peer ...`.
9. Os parâmetros WG são enviados ao UserAuthn pelo túnel Noise.
10. O UserAuthn escreve `wg0.conf` e executa `wg-quick up wg0` — o túnel VPN fica ativo.

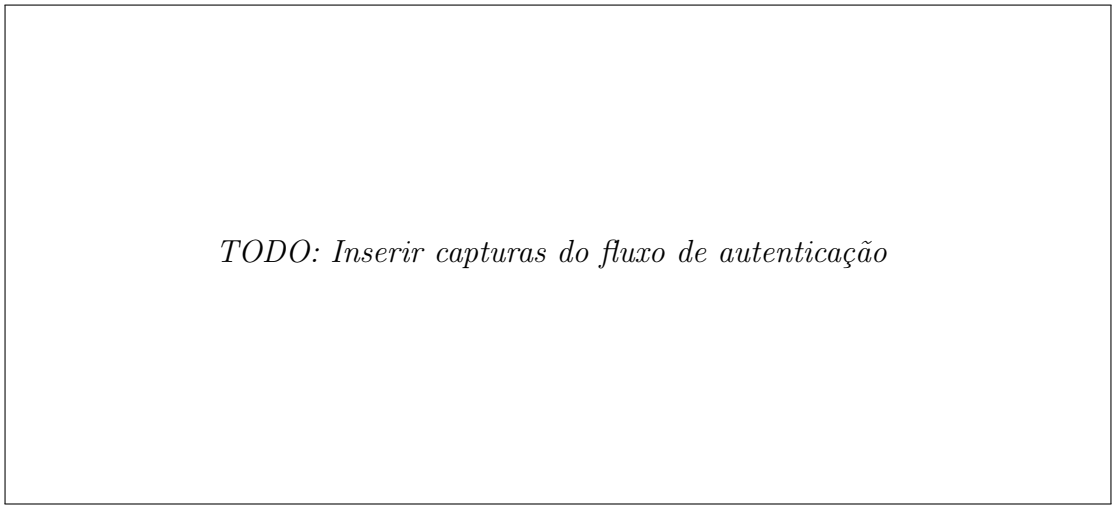


Figura 2: Fluxo de autenticação completo.

5 Túnel Noise-IK

A comunicação entre o UserAuthn e o SrvAuthn utiliza o protocolo **Noise-IK** (Noise Protocol Framework, revisão 34), implementado de raiz em `common/noise.py` sem recurso a bibliotecas Noise de terceiros. Os primitivos criptográficos são fornecidos pela biblioteca `cryptography` (hazmat):

- **Curve25519** — troca de chaves Diffie-Hellman.
- **ChaCha20-Poly1305** — cifra AEAD para o transporte.
- **BLAKE2s** — função de hash para o estado do handshake.

O padrão IK garante que o iniciador conhece a chave estática do responder *a priori* — no nosso caso, a chave pública WireGuard. As duas mensagens do *handshake*:

1. `-> e, es, s, ss` — o iniciador envia chave efémera, DH com estática do responder, a sua estática cifrada e DH mútuo.

2. `<- e, ee, se` — o responder responde com chave efémera e os DH restantes.

Após o *handshake*, cada direção tem o seu próprio `CipherState` (ChaCha20-Poly1305 com nonce incremental). Todas as mensagens subsequentes são frames com comprimento prefixado (`common/framing.py`: 4 bytes big-endian + payload, limite de 64 KiB).

6 Proxy HTTP local

Para que o browser não necessite de contactar diretamente o gateway (o que exigiria TLS e certificados), o UserAuthn implementa um proxy HTTP local:

- O `client/proxy.py` escuta em `127.0.0.1:18080`.
- Para cada ligação do browser, abre um *novo* túnel Noise-IK ao SrvAuthn com `{"service":"proxy"}` como primeiro frame.
- O `server/proxy.py` abre uma ligação TCP ao IdP em `localhost:8080` e faz *relay* bidirecional.

O `ORIGIN` do IdP é `http://localhost:18080`, pelo que o `verification_uri_complete` gerado aponta para o proxy. O browser abre esse URL; o tráfego flui sobre Noise-IK; o utilizador nunca vê o endereço real do gateway.

7 Fatores de autenticação (MFA)

Três fatores são implementados, por ordem crescente de robustez:

7.1 Palavra-passe — desafio-resposta

A palavra-passe nunca abandona o browser em texto claro. O protocolo:

1. **Registo (helpdesk):**
   - `salt = os.urandom(16)`
   - `verifier = scrypt(password, salt, N=`2^{14}`, r=8, p=1)` $\rightarrow$ 32 bytes
   - Armazena `(salt, verifier)`.
2. **Login (IdP):**
   - O servidor envia `(salt, nonce, parâmetros scrypt)`.
   - O browser calcula `verifier' = scrypt(password, salt)`.
   - O browser calcula `proof = HMAC-SHA256(verifier', nonce)`.
   - O servidor verifica: `expected = HMAC-SHA256(verifier, nonce)`, comparação em tempo constante.
   - O `nonce` é marcado como consumido atomicamente (`BEGIN IMMEDIATE`) — replay impossível.

O scrypt no browser é implementado em JavaScript puro (`idp/static/js/scrypt.js`), seguindo o RFC 7914.

7.2 TOTP

RFC 6238: código de 6 dígitos, janela de 30 segundos, tolerância de ±1 *step*. O segredo partilhado é armazenado **cifrado em repouso** com AES-256-GCM (secção 11).

7.3 FIDO2 / WebAuthn

Utiliza a biblioteca `webauthn` ≥ `2.0`. Tanto o registo (helpdesk) como a autenticação (IdP) exigem `UserVerificationRequirement.REQUIRED`, o que obriga o autenticador a realizar verificação local (PIN ou biometria). Uma chave de segurança roubada, sem PIN, não satisfaz este fator.

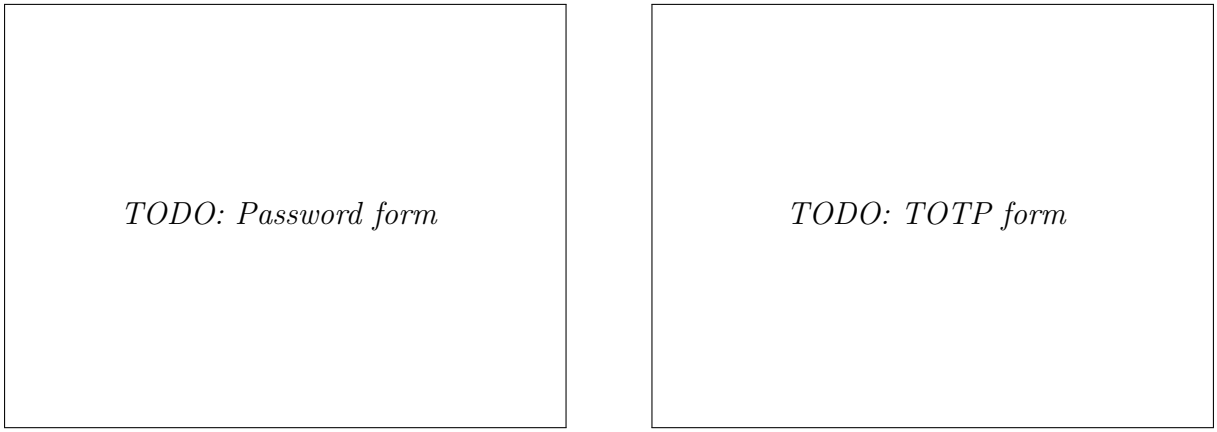


Figura 3: Formulários MFA: palavra-passe (esq.) e TOTP (dir.).

8 Orquestração PAM

A especificação exige que a orquestração MFA e RBA seja feita com PAM. A implementação segue esta arquitetura:

8.1 Módulo PAM customizado

O ficheiro `idp/pam_iaa/src/pam_vpnmfa.c` é compilado em `pam_vpnmfa.so` e instalado como módulo PAM. O perfil `/etc/pam.d/iaa-vpn` contém:

```
# PAM service for the IAA VPN MFA orchestrator
auth    required    pam_vpnmfa.so
```

Listing 1: Perfil PAM `iaa-vpn`

8.2 Funcionamento

1. O IdP calcula a pontuação de risco e define `IAA_RISK_SCORE` como variável de ambiente PAM.
2. O *PAM bridge* (`idp/pam_bridge.py`) invoca `pam_authenticate` numa thread dedicada.
3. O módulo C lê `IAA_RISK_SCORE`, determina o tier e emite mensagens `FACTOR:<nome>` pela função de conversação.

4. O Python bridge espera em `threading.Event`; o Flask renderiza o formulário correspondente e submete o resultado.

8.3 Contrato de conversação

Tabela 1: Protocolo de conversação PAM.

Estilo	Mensagem	Resposta esperada
`PAM_PROMPT_ECHO_OFF`	`FACTOR:password`	`OK` / `FAIL` / `UNAVAILABLE`
`PAM_PROMPT_ECHO_OFF`	`FACTOR:totp`	`OK` / `FAIL` / `UNAVAILABLE`
`PAM_PROMPT_ECHO_OFF`	`FACTOR:fido2`	`OK` / `FAIL` / `UNAVAILABLE`
`PAM_TEXT_INFO`	`INFO:tier=L2 ...`	(qualquer string)

8.4 Mapeamento de tiers

Tabela 2: Mapeamento de pontuação de risco para tiers e fatores.

Tier	Pontuação	Fatores exigidos
L1	< 30	Palavra-passe
L2	< 60	Palavra-passe + TOTP
L3	≥ 60	Palavra-passe + TOTP + FIDO2

9 Autenticação baseada em risco (RBA)

O motor de risco (`idp/rba.py`) agrega quatro sinais, todos obtidos a partir de dados locais na base de dados — sem chamadas a APIs externas:

Tabela 3: Sinais do motor de risco.

Sinal	Penalização	Origem
Primeiro login do utilizador	+0.30	`rba_ip_history` vazia
IP novo (utilizador existente)	+0.20	IP ausente de `rba_ip_history`
Falhas recentes (últimos 30 min)	+0.10 cada (máx. +0.40)	`rba_events`
Hora invulgar (00:00–06:00 UTC)	+0.20	`datetime.now().hour`

A pontuação final é limitada ao intervalo $[0, 1]$ e escalada para $[0, 100]$ para o módulo PAM. Se o tier exigido requerer um fator para o qual o utilizador não está inscrito, o sistema retorna erro 403 sem degradar silenciosamente para um tier inferior.

10 Camada de dados

Base de dados SQLite única em `/data/vpnauth.db` (modo WAL, foreign keys ativas), partilhada entre o IdP e o Helpdesk via *bind mount*.

Três operações utilizam `BEGIN IMMEDIATE` para evitar condições de corrida:

- **lease_next_ip** — alocação atómica de IP.
- **consume_password_nonce** — queima atómica do nonce de desafio-resposta.
- **mark_token_used** — marcação atómica de token OAuth como usado.

11 Robustecimento de segurança

Para além dos requisitos da especificação, foram implementadas as seguintes medidas de robustecimento:

Tokens OAuth hasheados em repouso
Armazenados como `SHA-256(token)`, TTL de 60 s, uso único (`used_at` atualizado atomicamente).

Bloqueio de conta
5 falhas em 5 minutos bloqueiam a conta durante 15 minutos. O contador reseta após login bem-sucedido.

Segredos TOTP cifrados em repouso
AES-256-GCM com chave mestra (`IAA_MASTER_KEY`) validada no arranque.

Arranque *fail-closed*
Todos os processos recusam arrancar com segredos em falta ou *placeholder*.

Alocação de IP livre de corridas
`BEGIN IMMEDIATE` no SQLite garante que dois provisionamentos concorrentes não obtêm o mesmo IP.

Validação de formato de chaves WG
A PSK do cliente é decodificada de base64 e verificada como exatamente 32 bytes antes de ser passada ao `wg set`.

Helpdesk apenas em loopback
O bind a `127.0.0.1` com `network_mode: host` garante que o helpdesk é inacessível externamente.

Expiração de leases
Thread de *sweeper* que remove peers expirados da interface `wg0`.

12 Testes e validação

12.1 Ambiente de teste

Os testes foram realizados com Docker Compose em máquina Linux (Ubuntu 24.04). O contentor `vpn-client` simula o host do utilizador com namespace de rede próprio, permitindo testar o túnel WireGuard completo numa única máquina.

12.2 Registo de utilizador (Helpdesk)

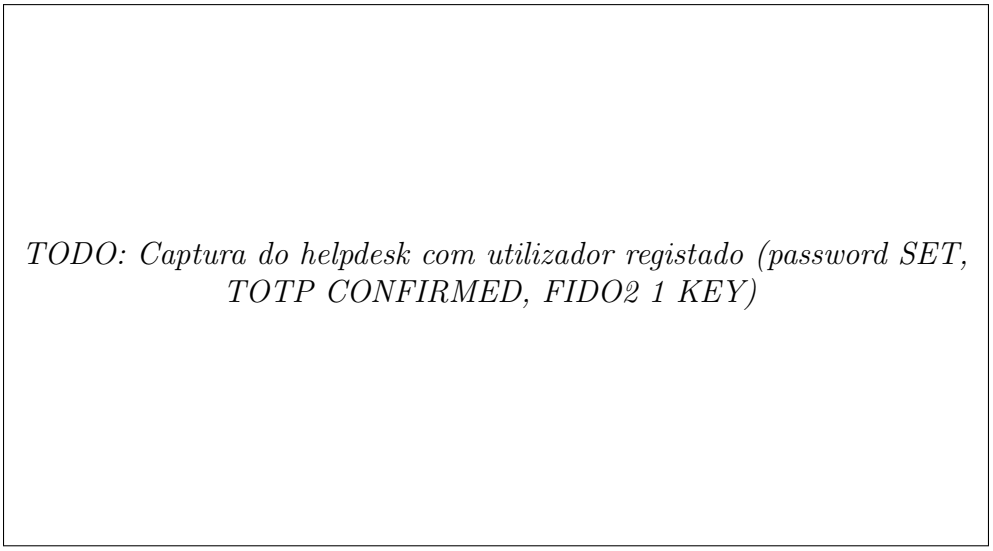


Figura 4: Interface do Helpdesk com utilizador totalmente inscrito.

12.3 Fluxo de autenticação completo

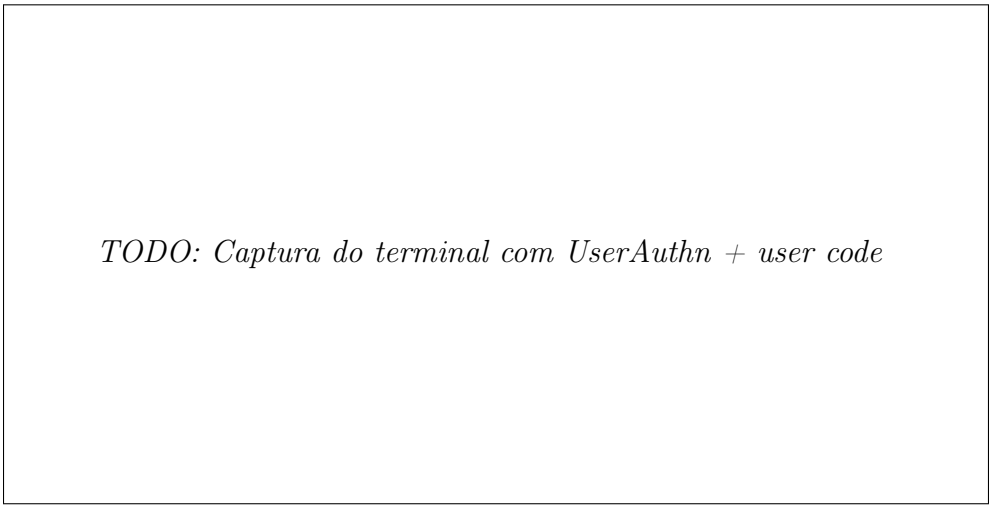


Figura 5: Terminal do UserAuthn a apresentar o código do dispositivo.

TODO: Captura do browser durante o fluxo MFA

Figura 6: Browser a completar a autenticação multi-fator.

12.4 Túnel WireGuard ativo

TODO: Captura de `wg show wg0` no cliente e servidor

Figura 7: Estado da interface WireGuard após autenticação.

TODO: Captura de `ping 10.10.10.1` bem-sucedido

Figura 8: Ping através do túnel VPN confirmando conectividade.

12.5 Tiers de risco diferentes

TODO: Login L1 (só password)

TODO: Login L3 (3 fatores)

Figura 9: Comparação de tiers: L1 (esq.) vs L3 (dir.).

12.6 Bloqueio de conta

TODO: Captura de conta bloqueada após 5 falhas

Figura 10: Mensagem de bloqueio de conta.

12.7 Dados cifrados em repouso

TODO: Queries à BD mostrando tokens hasheados, TOTP cifrado, verifier scrypt

Figura 11: Dados sensíveis armazenados cifrados/hasheados na base de dados.

13 Funcionalidades não implementadas

- **Indicador RBA "Autonomous Service" (AS/ASN).** A especificação lista *"same/different Autonomous Service"* como um dos sinais de exemplo para RBA. Não foi implementado porque requereria incluir uma base de dados GeoIP/ASN (∼50 MB) ou realizar chamadas a APIs externas em cada login — o que constitui uma fuga de privacidade e uma dependência externa num sistema cujo modelo de confiança é deliberadamente fechado. A linguagem da especificação é ilustrativa (*"consider using ... etc."*), e os restantes três indicadores nomeados estão implementados.

14 Conclusão

O sistema implementado cumpre todos os requisitos obrigatórios da especificação do projeto: autenticação de utilizadores WireGuard com três fatores MFA (palavra-passe por desafio-resposta, TOTP e FIDO2), orquestração via módulo PAM customizado, autenticação baseada em risco, e provisionamento dinâmico de parâmetros WireGuard sobre um fluxo OAuth 2.0 Device Authorization Grant.

A raiz de confiança é a chave pública WireGuard do gateway, utilizada simultaneamente como chave estática Noise-IK e como chave do plano de dados, eliminando a necessidade de TLS, certificados ou infraestrutura PKI.

Adicionalmente, foram implementadas medidas de robustecimento além do especificado: tokens hasheados e de uso único, bloqueio de conta, cifra de segredos TOTP em repouso, arranque *fail-closed* e expiração automática de leases.

15 Contribuições

Tabela 4: Contribuições dos autores.

Número	Nome	Contribuição
XXXXX	Nome Autor 1	XX%
XXXXX	Nome Autor 2	XX%
XXXXX	Nome Autor 3	XX%

16 Código de terceiros

Trabalho original: implementação Noise-IK (`common/noise.py`), scrypt no browser (`idp/static/js/scrypt.js`), módulo PAM (`pam_vpnmfa.c`), wrapper ctypes libpam (`common/libpam.p`) e todo o restante código em `client/`, `server/`, `idp/`, `common/` e `scripts/`.

Tabela 5: Bibliotecas externas utilizadas.

Biblioteca	**Utilização**
`flask`	Servidor HTTP para o IdP e helpdesk
`pyotp`	Geração e verificação TOTP (RFC 6238)
`webauthn`	Verificação WebAuthn / FIDO2
`qrcode, pillow`	Códigos QR para registo TOTP
`cryptography`	X25519, ChaCha20-Poly1305, AES-256-GCM, BLAKE2s

Ferramentas de sistema invocadas por subprocesso: `wg`, `wg-quick`, `wg genkey`, `wg pubkey`, `wg genpsk`.